

Companion to Bird & de Moor, Algebra of Programming

§2. Functions and Categories

§2.1. Categories

§2.2. Functors

§2.3. Natural transformations

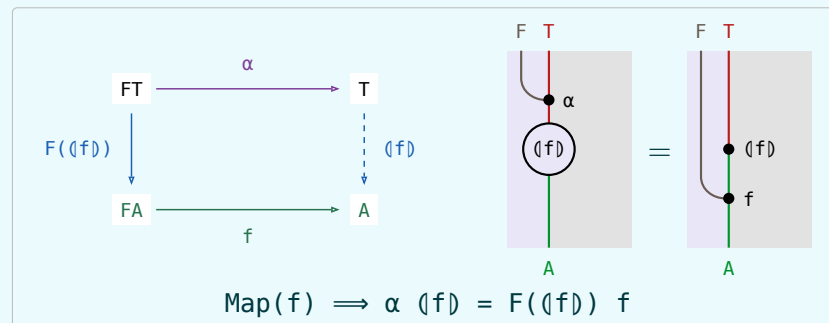
§2.4. Constructing datatypes

§2.5. Products and coproducts

§2.6. Initial algebras

(2.6a)

An **F-algebra** is a map $f : F(A) \rightarrow A$; A is its **carrier**. An **F-homomorphism** from $f : F(A) \rightarrow A$ to $g : F(B) \rightarrow B$ is a map $h : A \rightarrow B$ with $f \circ h = F(h) \circ g$. The **initial algebra** $\alpha : F(T) \rightarrow T$ is the F-algebra with exactly one F-homomorphism $\llbracket f \rrbracket : T \rightarrow A$ to every F-algebra f .



(2.6b)

$\alpha \ h = F(h) \ [c, f] \iff \text{zero } h = c \wedge \text{succ } h = h \ f$

h is a homomorphism from $\alpha = [\text{zero}, \text{succ}]$ to $[c, f]$ exactly when h sends **zero** to **c** and **succ** then h equals h then **f**

the right side

(1) (2) (3)

1	$F(h) \ [c, f]$	definition of F
2	$= (1 + h) \ [c, f]$	coproduct
3	$= [c, h \ f]$	

the left side

(1) (2) (3)

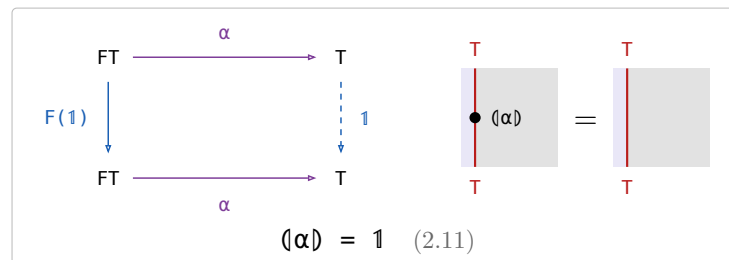
1	$\alpha \ h$	since $\alpha = [\text{zero}, \text{succ}]$
2	$= [\text{zero}, \text{succ}] \ h$	coproduct
3	$= [\text{zero } h, \text{succ } h]$	cancellation, zero arm

(4) (5)

4	$\iff [\text{zero } h, \text{succ } h] = [c, h \ f] \implies \text{zero } h = c$	cancellation, SUCC arm
5	$\text{and } [\text{zero } h, \text{succ } h] = [c, h \ f] \implies \text{succ } h = h \ f$	

§2.6.1. Reflection

(2.6.1a)

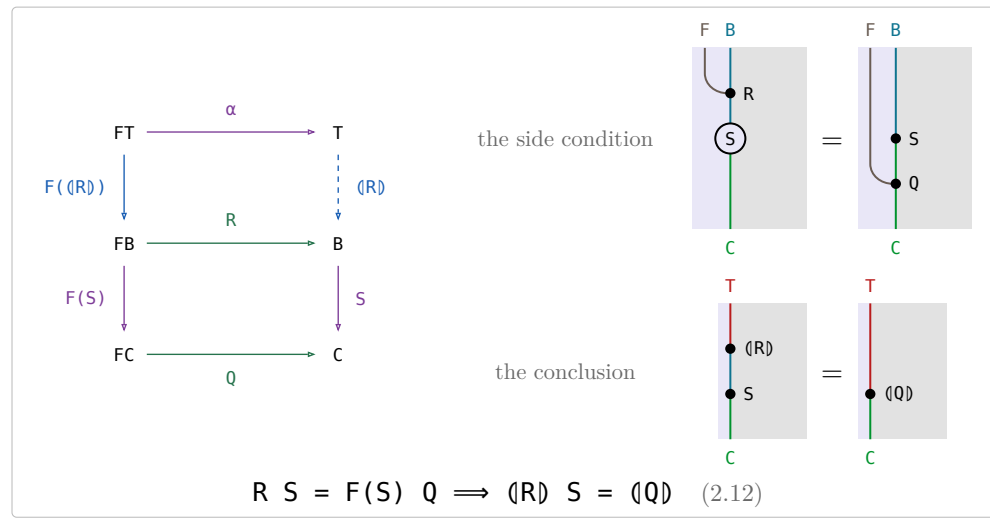


Taking a value apart with α and putting it straight back is doing nothing.

§2.6.2. $R \ S = F(S) \ Q \implies (R) \ S = (Q)$ — folding with R , then applying S , is folding with Q

When $S : B \rightarrow C$ is an F -homomorphism from $R : F(B) \rightarrow B$ to $Q : F(C) \rightarrow C$, folding with R and then applying S is folding with Q .

(2.6.2a)



§2.7. Type functors

(2.7a)

Let F be a bifunctor taking both the parameter A and the recursive position TA , with an initial algebra $\alpha_A : F(A, TA) \rightarrow TA$ for every object A . Then T is a functor, acting on a map $f : A \rightarrow B$ by

$$T(R) \triangleq (F(R, 1) \alpha)$$

(2.7b)



- $F : \mathcal{A} \times \mathcal{A} \rightarrow \mathcal{A}$ is a bifunctor and a wire is a unary functor, so the two arguments are packed first: $\langle 1, T \rangle : \mathcal{A} \rightarrow \mathcal{A} \times \mathcal{A}$ sends A to (A, TA) , and $F(\langle 1, T \rangle(A))$ is $F(A, TA)$.
- The picture is three wires — F , $\langle 1, T \rangle$, and the object — and the region between the first two is $\mathcal{A} \times \mathcal{A}$.
- α is then an ordinary natural transformation $F \circ \langle 1, T \rangle \Rightarrow T$: its bead eats the F and $\langle 1, T \rangle$ wires, and the T wire is born under it.
- $F(f, T(f))$ costs no notation. It is the bead f on the object wire with $\langle 1, T \rangle$ and F running past: $\langle 1, T \rangle$ is what turns f into the pair $(f, T(f))$, and F is what applies it.
- The law is the naturality of α , which is exactly the freedom to slide that f bead past it.

(2.7c)



(2.7d)

	name	law	what it says
1	functor	$T(1) = 1$ and $T(R) T(S) = T(R S)$	Acting by the identity changes nothing, and two actions in a row are one action.
2	type functor fusion	$T(f) (fh) = (F(f, 1) h)$	A relator action followed by a fold is a single fold — the intermediate structure is never built. The side condition holds because F is a bifunctor — $F(R, 1)F(1, (Q)) = F(R, (Q)) = F(1, (Q))F(R, 1)$.
3	naturality of α	$\alpha T(f) = F(f, T(f)) \alpha$	Building and then mapping is the same as mapping the parts and then building, so α is natural from $G(R) = F(R, T(R))$ to T .